برنامه نویسی پیشرفته

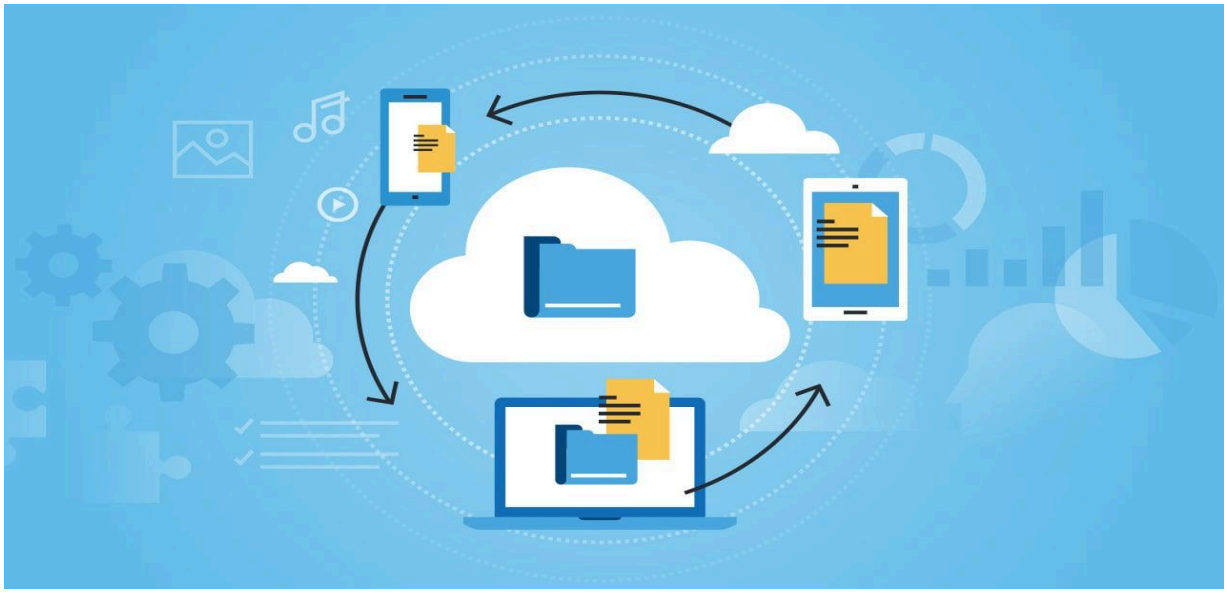
زمستان ۱۴۰۲ و بهار ۱۴۰۳ - دانشکده علوم ریاضی
دانشگاه صنعتی شریف

تمرین سری پنجم

زمان تحویل: – 8 تیرماه

مقدمه و اهداف تمرین:

در این تمرین قصد داریم با مباحث مولتی ترد و شبکه آشنا بشیم و می‌خواهیم نسخه‌ای ساده‌تر از سرویس‌های ذخیره‌سازی ابری مانند Google Drive یا Dropbox را در زبان برنامه‌نویسی جاوا پیاده‌سازی کنید. این سرویس به کاربران امکان می‌دهد تا فایل‌های خود را بر روی سرور آپلود کرده و آن‌ها را مدیریت و بازیابی کنند.



نحوه پیاده‌سازی تمرین:

در این تمرین، دو پروژه جداگانه برای کلاینت و سرور باید پیاده‌سازی شود. ابتدا سرور راه‌اندازی می‌شود و منتظر اتصال کلاینت‌ها می‌ماند. پس از برقراری اتصال، کلاینت باید بتواند وارد حساب کاربری خود شده و یا حساب کاربری جدید ایجاد کند. سپس کاربر باید بتواند فایل‌های آپلود شده از قبل را مشاهده کرده و بر اساس انتخاب خود، آن‌ها را دانلود نماید. همچنین، امکان انتخاب فایل جدید و آپلود آن به سرور باید وجود داشته باشد. ارتباط بین کلاینت و سرور از طریق درخواست‌های زیر انجام می‌شود: درخواست ایجاد حساب کاربری و ورود به حساب- درخواست نمایش لیست فایل‌های آپلود شده - درخواست دانلود فایل - درخواست آپلود فایل- درخواست دسترسی به یک فایل مشخص (امتیازی)

الگوریتم دانلود و آپلود

Parallel Downloading & Uploading:

پس از انتخاب فایل مورد نظر برای دانلود یا آپلود، ابتدا حجم فایل بررسی می‌شود و سپس فایل به قسمت‌های مساوی تقسیم می‌شود. به عنوان مثال، اگر فایل 100 مگابایتی باشد به 5 قسمت تقسیم می‌شود که هر قسمت 20 مگابایت خواهد بود. سپس برای هر قسمت یک ترد جداگانه ایجاد می‌شود و هر ترد به طور مستقل فرایند دانلود یا آپلود را انجام می‌دهد. همچنین، در حین این عملیات از یک نشانگر پیشرفت (progressbar) استفاده می‌شود تا وضعیت پیشرفت به کاربر نمایش داده شود. پس از اتمام کار هر ترد، فایل کامل می‌شود و تردهای مربوطه بسته خواهد شد. تعداد تردها برابر با تعداد قسمت‌هایی است که فایل به آن تقسیم شده است و این تعداد بسته به اندازه فایل و سرعت ارتباط شبکه محاسبه می‌شود.

رمزگذاری و Hashing رمز عبور

پس از دریافت اطلاعات مورد نیاز توسط کاربر برای ساخت اکانت (نام کاربری و رمز عبور) باید این اطلاعات در سرور ذخیره شود. حال برای پسورد به دلایل امنیتی نباید رمز عبور را به صورت متنی ساده ذخیره کنیم. در عوض، باید از یک الگوریتم هشینگ قوی استفاده کنیم تا رمز عبور به صورت یک مقدار هش شده ذخیره شود. این کار باعث می‌شود حتی در صورت دسترسی غیر مجاز به پایگاه داده، کسی نتواند به رمزهای عبور واقعی دسترسی پیدا کند. می‌تونید برای انجام این کار از کتابخانه‌های خارجی نیز استفاده کنید.

رابط کاربری

برای استفاده از برنامه، یک رابط کاربری بصورت کنسول استاندارد (CLI) یا یک رابط گرافیکی (امتیازی) طراحی کنید که به کاربر امکانات زیر را ارائه دهد:

- ساخت و ورود به اکانت: کاربر باید بتواند حساب کاربری جدیدی ایجاد کند یا به حساب کاربری خود وارد شود.
- انتخاب فایل برای آپلود: کاربر باید قادر باشد فایلی را انتخاب کند تا به سرور آپلود شود.
- مشاهده فایل‌های آپلود شده در سرور: کاربر باید لیستی از فایل‌های آپلود شده از قبل را در سرور مشاهده کند.
- انتخاب فایل برای دانلود: کاربر باید بتواند یک فایل را از لیست فایل‌های موجود برای دانلود انتخاب کند.
- درخواست دسترسی (اختیاری): کاربر باید بتواند درخواست دسترسی به یک فایل مشخص را ارسال کند و همچنین درخواست‌های ارسال شده را مشاهده، رد و قبول کند.
- این رابط کاربری به کاربر امکان مدیریت حساب و فایل‌های خود را بدهد و از طریق آن بتواند به راحتی از امکانات برنامه استفاده کند.

نکات:

1. ارتباط بین کلاینت و سرور صرفا باید از طریق شبکه باشد.
2. برای برقراری ارتباط بین سرور و کلاینت، از پروتکل‌های UDP و TCP استفاده کنید. برای فرایند ساخت اکانت و لاگین از TCP و برای بقیه بخش‌ها از UDP استفاده کنید.
3. رعایت اصول کدنویسی تمیز در این تمرین اجباریست.
4. فایل‌های آپلود شده مربوط به هر اکانت باید فقط و فقط در سرور ذخیره شوند و هر فایل آپلود شده باید شناسه‌ای (ID) منحصر به فرد داشته باشد.
5. استفاده از کلاس‌های آماده جاوا برای موضوعات همروندی و از پکیج java.util.concurrent ممنوع است.
6. دقت داشته باشید تمامی فایل های آپلود شده و یوزر های ساخته شده باید در سرور ذخیره شوند تا در استفاده های بعدی بتوان از آن‌ها استفاده کرد.

بخش امتیازی:

دسترسی یا Access دادن:

دسترسی به این صورت است که شما نام یا آی‌دی فایلی رو وارد میکنید تا درخواست دسترسی (AccessRequest) به سرور ارسال شود و سرور به کاربری که فایل متعلق به آن است درخواست تایید (AcceptRequest) ارسال می کند و با توجه به تایید یا رد کاربر Response های لازم ارسال می‌شود. در صورت تایید دسترسی، فایل درخواستی که متعلق به کاربری دیگر است به نمایش در می‌یاید و قابل دانلود است. هر یوزر باید بتواند لیست کاربرانی که به هر فایل دسترسی دارند و ببینه و در صورت لزوم دسترسی آن را قطع کند.

دانلود و آپلود همزمان:

کاربر باید بتواند به صورت همزمان چندین فایل را آپلود و دانلود کند (الگوریتم دانلود و آپلود باید رعایت شود)

JSON Web Tokens:

احراز هویت با توکن‌های JWT یکی از روش‌های موثر برای اطمینان از اعتبار و هویت کاربران در برنامه‌های شبکه است. در این روش، سرور پس از موفقیت آمیز بودن فرایند ساخت اکانت، یک توکن JWT برای کاربر ایجاد می‌کند که شامل اطلاعات هویتی کاربر و تاریخ انقضای آن است. سپس توکن JWT به کاربر ارسال می‌شود تا او بتواند با استفاده از آن به سرور و منابع مورد نیاز خود دسترسی پیدا کند. کلاینت باید توکن JWT را به هر درخواستی که به سرور ارسال می‌کند، اضافه کند تا سرور بتواند اعتبار آن را بررسی کرده و به درستی عمل کند. سپس سرور توکن را بررسی کرده و اعتبار آن را تأیید می‌کند، از جمله بررسی امضای توکن، اعتبار زمانی و سایر اطلاعات مورد نیاز. پس از تأیید اعتبار توکن، سرور دسترسی کاربر به منابع مورد نیاز را تأیید کرده و عملیات مورد نیاز را انجام می‌دهد.